Quantum Krylov subspace methods

A short note for numerical linear algebraists

Martin Stoll

November 2026

Abstract

Quantum Krylov methods compute the lowest eigenvalue of a huge Hermitian matrix by running a Rayleigh–Ritz procedure. The quantum computer supplies the entries of the small projected matrices and a classical computer solves the resulting generalized eigenvalue problem. This note explains the idea in standard linear algebra notation. It covers what a quantum device can and cannot do, which Krylov bases it can generate, and why the main numerical difficulty is an ill-conditioned Gram matrix whose entries carry statistical noise. A small experiment illustrates the points, and we list open questions where numerical linear algebra can contribute.

1 Motivation

Many problems in quantum physics and chemistry reduce to computing the smallest eigenvalue λ_0 of a Hermitian matrix $H \in \mathbb{C}^{n\times n}$. Physicists call H the *Hamiltonian*, λ_0 the *ground state energy* and its eigenvector u_0 the *ground state*. For a system of N particles that each have two possible configurations (spins, qubits, spin orbitals), H acts on $\mathbb{C}^2 \otimes \cdots \otimes \mathbb{C}^2$, so $n = 2^N$. For $N = 50$, a single vector in double precision already takes 16 petabytes. Lanczos, Davidson and their relatives are the methods of choice for moderate N. Beyond about $N \approx 50$, any method that stores vectors explicitly is hopeless, and one turns to compressed formats such as tensor networks.

A quantum computer with N qubits stores a vector of length 2^N *implicitly* and can multiply it by certain unitary matrices cheaply. The catch is that the vector can never be read out. One only gets random samples whose statistics encode quadratic forms. Quantum Krylov methods (also called quantum subspace diagonalization) are a pragmatic way of combining this ability with the Rayleigh–Ritz principle [1–7]. Experiments on current hardware now reach subspace dimensions and problem sizes of practical interest [8]. For an NLA reader these methods are attractive because their core difficulties are classical: a non-orthogonal Krylov basis, an ill-conditioned Gram matrix, and a generalized eigenvalue problem with noisy data.

2 A quantum computer as a restricted linear algebra device

This note assumes no background beyond linear algebra. This section translates the quantum computing vocabulary into statements about vectors and matrices. Throughout, N is the number of qubits and $n = 2^N$.

2.1 Vectors

The memory of a quantum computer with N qubits holds a single unit vector $v \in \mathbb{C}^n$, called the *state*. One qubit holds a unit vector in $\mathbb{C}^2$, and N qubits hold a vector in $\mathbb{C}^2 \otimes \cdots \otimes \mathbb{C}^2 = \mathbb{C}^n$.

The standard basis vectors are indexed by bit strings,

$$e_{b_1 b_2 \ldots b_N} = e_{b_1} \otimes e_{b_2} \otimes \cdots \otimes e_{b_N}, \qquad e_0 = \begin{bmatrix} 1 \\ 0 \end{bmatrix},\ e_1 = \begin{bmatrix} 0 \\ 1 \end{bmatrix},$$

so the index of an entry of v is simply its binary representation. A *product state* is a Kronecker product $a_1 \otimes \cdots \otimes a_N$ of unit vectors $a_i \in \mathbb{C}^2$. It is described by $2N$ numbers, whereas a general v needs $n = 2^N$. Physicists write $|v\rangle$ for v and $\langle w|v\rangle$ for $w^* v$. We do not use this notation.

The device always starts from $e_{0\ldots0}$, and all further vectors are obtained by multiplying with unitary matrices. A start vector v_0 is therefore available only if $v_0 = Q_0 e_{0\ldots0}$ for a unitary Q_0 that can be applied cheaply in the sense of Section 2.3.

2.2 The matrix

The $n \times n$ matrix H is never stored. It is given as a sum of $L = \mathrm{poly}(N)$ *Pauli strings*,

$$H = \sum_{\ell=1}^{L} \alpha_\ell P_\ell, \qquad P_\ell = \sigma_{\ell,1} \otimes \cdots \otimes \sigma_{\ell,N}, \tag{1}$$

where $\alpha_\ell \in \mathbb{R}$ and each factor is one of the 2×2 matrices

$$I, \quad X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad Y = \begin{bmatrix} 0 & -\mathrm{i} \\ \mathrm{i} & 0 \end{bmatrix}, \quad Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}.$$

Storing H thus needs only $O(LN)$ numbers. Each P_ℓ is Hermitian and unitary. A standard example, used below, is the transverse-field Ising chain (a simple model of interacting magnetic moments)

$$H = -\sum_{i=1}^{N-1} Z_i Z_{i+1} - h \sum_{i=1}^{N} X_i, \qquad Z_i = I \otimes \cdots \otimes I \otimes Z \otimes I \otimes \cdots \otimes I \ (Z \text{ in position } i),$$

and X_i is defined in the same way. H is not unitary, so it cannot be applied directly. Section 2.3 explains what can be applied instead.

2.3 Which unitary matrices can be applied

Gates and circuits. The hardware applies a small set of elementary unitaries, the *gates*. A one-qubit gate is

$$I_{2^{i-1}} \otimes u \otimes I_{2^{N-i}}, \qquad u \in \mathbb{C}^{2\times 2} \text{ unitary},$$

i.e. it acts on the i-th Kronecker factor only. A two-qubit gate is the same with a 4×4 unitary w acting on two neighbouring factors. Applying one gate takes a fixed physical time, independent of n. The device thus performs a "matvec" with a $2^N \times 2^N$ matrix in one step, but only for these very special matrices. A *circuit* is a product $Q = G_g \cdots G_2 G_1$ of gates, and its cost is the number g of gates. Gates on disjoint factors commute and can be applied simultaneously. The number of such parallel layers is the *depth*, which determines the run time.

An important construction is the *controlled* version of a unitary $U \in \mathbb{C}^{n\times n}$,

$$C(U) = \begin{bmatrix} I_n & 0 \\ 0 & U \end{bmatrix} = e_0 e_0^T \otimes I_n + e_1 e_1^T \otimes U \in \mathbb{C}^{2n\times 2n}. \tag{2}$$

It uses one extra qubit (the first Kronecker factor) and applies U to the lower half of the vector only. If U has a circuit, $C(U)$ has one with a constant factor more gates. The simplest example is the permutation matrix $C(X) \in \mathbb{C}^{4\times4}$ (the CNOT gate).

unitary matrix	remark	cost (gates)
$u_1 \otimes \cdots \otimes u_N$, $u_i \in \mathbb{C}^{2\times 2}$	one gate per factor; includes all Pauli strings P_ℓ	N, depth 1
$\operatorname{diag}\left(e^{\mathrm{i}\varphi(b)}\right)_b$	φ computable by a short classical (Boolean) circuit	poly(N)
permutation $e_b \mapsto e_{\pi(b)}$	π computable by a short classical circuit, e.g. $b \mapsto b+1 \bmod n$	poly(N)
$e^{-\mathrm{i}\theta P}$, P a Pauli string	$P = QDQ^*$ with Q a Kronecker product and D diagonal (see text)	$O(N)$
unitary DFT matrix F_n	$(F_n)_{jk} = \omega^{jk}/\sqrt{n}$, $\omega = e^{2\pi\mathrm{i}/n}$ (quantum Fourier transform)	$O(N^2) = O(\log^2 n)$
$C(U)$	block diagonal, see (2)	const $\times$ cost(U)
$e^{-\mathrm{i}tH}$, H as in (1)	approximately, by products of $e^{-\mathrm{i}\theta P_\ell}$ (see text)	poly($N, t, 1/\text{accuracy}$)
U^k	no shortcut in general	$k\times$ cost(U)
block encoding of A	non-unitary A, applied with probability < 1 (see text)	depends on A

Table 1: Unitary matrices of size $n = 2^N$ that a quantum computer can apply cheaply.

What is cheap. Every unitary in $\mathbb{C}^{n\times n}$ is a product of gates. However, a generic unitary needs on the order of $n^2 = 4^N$ gates, because the unitary matrices form a set of real dimension n^2 and each gate contributes at most 16 real parameters. "Can be applied" therefore means in practice: a circuit with polynomially many gates in N, i.e. polylogarithmically many in n. These matrices are extremely structured. They resemble classical fast transforms, but the gap is far larger: the FFT costs $O(n \log n)$ flops, whereas the corresponding quantum circuit needs only $O(\log^2 n)$ gates. Table 1 lists the families relevant for this note.

Exponentials of Pauli strings. Each 2×2 Pauli matrix is diagonalized by a fixed 2×2 unitary: $X = hZh^*$ and $Y = (sh)Z(sh)^*$ with $h = \frac{1}{\sqrt{2}}\left[\begin{smallmatrix} 1 & 1 \\ 1 & -1 \end{smallmatrix}\right]$ and $s = \operatorname{diag}(1, \mathrm{i})$. Hence every Pauli string satisfies $P = QDQ^*$, where Q is a Kronecker product of 2×2 unitaries and D is a Kronecker product of I's and Z's. D is diagonal with entries ± 1, namely the parity of the bits of b in the positions where D has a Z. Therefore $e^{-\mathrm{i}\theta P} = Qe^{-\mathrm{i}\theta D}Q^*$, and $e^{-\mathrm{i}\theta D}$ is a diagonal matrix of the type in the second row of Table 1.

Time evolution. Each single term $e^{-\mathrm{i}t\alpha_\ell P_\ell}$ is cheap, but the matrices P_ℓ do not commute, so $e^{-\mathrm{i}tH} \neq \prod_\ell e^{-\mathrm{i}t\alpha_\ell P_\ell}$. For two Hermitian matrices A, B and a step $\delta > 0$ one has

$$\left\| e^{-\mathrm{i}\delta A} e^{-\mathrm{i}\delta B} - e^{-\mathrm{i}\delta(A+B)} \right\|_2 \leq \tfrac{\delta^2}{2} \left\| AB - BA \right\|_2,$$

so splitting the exponential is accurate only for *short* times. The remedy is to cut t into r steps of length $\delta = t/r$. Since $e^{-\mathrm{i}tH} = \left(e^{-\mathrm{i}(t/r)H}\right)^r$ exactly, one only has to split each short step:

$$e^{-\mathrm{i}tH} \approx \Big(\prod_{\ell=1}^{L} e^{-\mathrm{i}(t/r)\alpha_\ell P_\ell} \Big)^r \qquad \text{(Lie–Trotter)}.$$

Each of the r factors has error $O\big((t/r)^2\big)$. For unitaries, errors add up at most linearly: $\|U_1^r - U_2^r\|_2 \leq r\,\|U_1 - U_2\|_2$ (telescoping sum). The total error is therefore $O(t^2/r)$, with a constant involving the commutators of the P_ℓ, and it can be made small by increasing r. The circuit consists of Lr Pauli exponentials, so accuracy ε costs $r = O(t^2/\varepsilon)$ steps. Higher-order product formulas and other methods reduce the cost to roughly linear in t. For a general H the cost of

$e^{-\mathrm{i}tH}$ *must* grow linearly with t (this is the "no fast-forwarding" theorem). Classically, H^k or U^k are free once H is diagonalized; here U^k costs k times as much as U. In a Krylov method, the last basis vector is therefore the most expensive one.

Block encodings: using non-unitary matrices. A non-unitary $A \in \mathbb{C}^{n\times n}$ with $\|A\|_2 \le \alpha$ can be used through a larger unitary

$$W = \begin{bmatrix} A/\alpha & * \\ * & * \end{bmatrix} \in \mathbb{C}^{2^a n \times 2^a n},$$

which needs a extra qubits. The blocks $*$ are whatever makes W unitary. A unitary completion always exists, e.g. for $a = 1$ and $B = A/\alpha$ (so $\|B\|_2 \le 1$)

$$W = \begin{bmatrix} B & (I - BB^*)^{1/2} \\ (I - B^*B)^{1/2} & -B^* \end{bmatrix}.$$

What matters is that W has a short circuit. For H as in (1) with $\alpha = \sum_\ell |\alpha_\ell|$, such a W can be built from $O(L)$ controlled Pauli strings. We write $[x; y]$ for the vector with block x on top of block y. Applying W to $[v; 0]$ gives $[Av/\alpha;\, w]$. Measuring the extra qubits (Section 2.4) yields the top block with probability $\|Av\|^2/\alpha^2$, and the state is then $Av/\|Av\|$. Otherwise the run is discarded. So A is applied only with some probability, and the result is always normalized. Products of W, W^* and simple diagonal unitaries give block encodings

$$\begin{bmatrix} p(A/\alpha) & * \\ * & * \end{bmatrix}$$

of polynomials p of degree d with $|p(x)| \le 1$ on $[-1, 1]$ (for Hermitian A), at the cost of about d applications of W [9]. For the Chebyshev polynomials T_k this is particularly simple.

What is not possible.

- Applying a generic dense unitary, which needs exponentially many gates.
- Applying H, or any non-unitary matrix, deterministically. Block encodings apply it only with some probability, and the result is normalized.
- Nonlinear operations on the vector: normalization, Gram–Schmidt, dividing by an inner product, or any step whose choice depends on the entries of v (other than through measurement).
- Copying: no unitary Q satisfies $Q(v \otimes e_{0\ldots0}) = v \otimes v$ for all v, since $v \mapsto v \otimes v$ is not linear (the "no-cloning" theorem).
- Reading out v. Section 2.4 describes what can be extracted.

Hardware today. Each two-qubit gate is faulty with probability around 10^{-3}, so circuits with more than about 10^3–10^4 two-qubit gates produce mostly noise. Moreover, gates usually act only on physically neighbouring qubits. Future *error-corrected* hardware will store each qubit redundantly in many physical qubits and correct errors on the fly, which allows much longer circuits at a large overhead in qubits.

	classical Lanczos	quantum Krylov
storage of v	explicit, $O(n)$	implicit, $N = \log_2 n$ qubits
basis generator	$v \mapsto Hv$	$v \mapsto f(H)v$, $f(H)$ unitary or block encoded
inner products	exact up to rounding	sampled, error $\sim N_s^{-1/2}$
orthogonalization	Gram–Schmidt (Lanczos recurrence)	impossible
projected problem	tridiagonal T_m	dense pencil (M, G), G ill-conditioned

Table 2: Lanczos versus quantum Krylov methods.

2.4 Output: measurement and inner products

Measurement. Measurement is the only way to obtain information from the device. Measuring all qubits of the state v returns an index b with probability $|v_b|^2$ (these sum to one because $\|v\| = 1$) and replaces v by e_b. Measuring only the first qubit of $x = [x_0; x_1]$, $x_0, x_1 \in \mathbb{C}^n$, returns 0 with probability $\|x_0\|^2$, and the state becomes $x_0/\|x_0\|$, or returns 1 with probability $\|x_1\|^2$, and the state becomes $x_1/\|x_1\|$. Since one only gets samples, every number must be estimated from many repetitions, and each repetition must prepare the vector again.

The Hadamard test estimates v^*Wv for a unitary W with a circuit. It uses one extra qubit, so all vectors lie in $\mathbb{C}^{2n}$, and h is the 2×2 matrix from above.

1. Prepare $[v; 0]$ and apply $h \otimes I_n$: $\quad \frac{1}{\sqrt{2}}[v;\, v]$.
2. Apply $C(W)$ from (2): $\quad \frac{1}{\sqrt{2}}[v;\, Wv]$.
3. Apply $h \otimes I_n$: $\quad \frac{1}{2}[v + Wv;\, v - Wv]$.
4. Measure the extra qubit. The outcome is 0 with probability $\frac{1}{4}\|v + Wv\|^2 = \frac{1}{2}\big(1 + \mathrm{Re}(v^*Wv)\big)$.

With $s = +1$ for outcome 0 and $s = -1$ for outcome 1, we get $\mathbb{E}[s] = \mathrm{Re}(v^*Wv)$. Starting step 2 instead from $\frac{1}{\sqrt{2}}[v;\, -\mathrm{i}v]$ gives $\mathrm{Im}(v^*Wv)$. Averaging N_s runs (*shots*) gives an estimate with standard deviation at most $N_s^{-1/2}$, so an accuracy of 10^{-3} costs about 10^6 runs. Hardware errors add further, partly systematic, perturbations. All quantities needed by the Krylov methods below have the form $v_0^*Wv_0$, with $W = U^k$ or $W = P_\ell U^k$.

Table 2 summarizes the consequences for a Krylov method. The two key restrictions are that inner products are noisy and that *no orthogonalization* is possible, since Gram–Schmidt needs the vectors themselves.

3 Rayleigh–Ritz with a non-orthogonal basis

Let $v_0, \ldots, v_{m-1}$ be vectors that can be prepared on the device and let $V = [v_0, \ldots, v_{m-1}] \in \mathbb{C}^{n \times m}$. V is never formed. We only need the $m \times m$ matrices

$$G = V^*V, \qquad M = V^*HV, \qquad G_{jk} = v_j^* v_k, \quad M_{jk} = v_j^* H v_k, \tag{3}$$

which the quantum computer estimates entry by entry. A classical computer then solves the generalized Hermitian eigenvalue problem

$$Mc = \theta\, Gc, \tag{4}$$

and the smallest Ritz value $\theta_{\min}$ approximates λ_0, with Ritz vector Vc. In exact arithmetic, and with G positive definite, this is standard Rayleigh–Ritz on range(V). By Courant–Fischer,

$\theta_{\min} \geq \lambda_0$, and $\theta_{\min}$ decreases monotonically as the subspace grows. Classically one would orthonormalize V first. Here that is impossible, so (4) is the generalized eigenvalue problem in the original, typically very ill-conditioned, basis. Everything that follows is about which vectors v_k to choose and how to cope with G.

The Ritz vector itself is not available, only its coefficient vector c. Other quantities, such as expectation values x^*Ax of observables A in the Ritz vector $x = Vc$, require further matrices V^*AV estimated in the same way.

4 Krylov bases that a quantum computer can generate

4.1 The role of $f(H)$

A quantum computer cannot apply H, so the basis vectors are $v_k = f_k(H)v_0$ for functions f_k that the device can implement, typically powers $f_k = f^k$ of one unitary $f(H)$. Here $f(H)$ does *not* approximate the product Hv. It replaces the matrix–vector product as the operation that *generates the subspace*, and H itself enters only through the projected matrix $M = V^*HV$ in (3). In a Krylov method the matvec only has to supply new directions rich in the wanted eigenvector. Write $H = \sum_i \lambda_i u_i u_i^*$ with $\lambda_0 \leq \lambda_1 \leq \cdots$ and $v_0 = \sum_i \gamma_i u_i$. Every $f_k(H)$ has the same eigenvectors as H, so

$$v_k = f_k(H)v_0 = \sum_i f_k(\lambda_i)\gamma_i u_i,$$

and Rayleigh–Ritz with H extracts the eigenvalues from whatever space the f_k produce. This is the familiar situation of shift-and-invert Lanczos, where $(H - \sigma I)^{-1}$ builds the space but one wants eigenvalues of H, or of polynomially filtered subspace iteration. Every vector in the subspace has the form $\phi(H)v_0$ with $\phi \in \operatorname{span}\{f_0, \ldots, f_{m-1}\}$. The quality of the space is determined by how well this function space contains filters ϕ that are large at λ_0 and small on the rest of the spectrum. The choice of the f_k also fixes the structure of G and M, and therefore how many numbers the device has to estimate. The two main choices behave quite differently.

- If f_k is a polynomial of degree k, e.g. $f_k(\lambda) = T_k(\lambda/\alpha)$ (Section 4.3), then $\operatorname{range}(V) = \mathcal{K}_m(H, v_0)$ is exactly the Lanczos space, only in a different basis.
- If $f_k(H) = U^k$ with $U = e^{-\mathrm{i}\Delta t H}$ (Section 4.2), the space consists of trigonometric polynomials in H applied to v_0. It is *not* the Lanczos space, and $U^k v_0$ is not an approximation of $H^k v_0$. The two are related only in the limit $\Delta t \to 0$. Since $U = I - \mathrm{i}\Delta t H + O(\Delta t^2)$, the scaled differences $\Delta t^{-j}(U - I)^j v_0$ tend to $(-\mathrm{i}H)^j v_0$, and $\mathcal{K}_m(U, v_0) \to \mathcal{K}_m(H, v_0)$. In this limit the vectors $U^k v_0$ become nearly identical, so G becomes singular. A practical $\Delta t \approx \pi/\|H\|_2$ is therefore a deliberate move away from the Lanczos space: one gets a different space, still containing good filters, whose basis $U^k v_0$ is far better conditioned than for small Δt (though still ill-conditioned, see Section 5).

4.2 Real-time Krylov

The most popular choice uses powers of the time evolution operator,

$$v_k = U^k v_0 = e^{-\mathrm{i}k\Delta t H} v_0, \qquad k = 0, \ldots, m-1, \tag{5}$$

i.e. $\operatorname{range}(V) = \mathcal{K}_m(U, v_0)$, a Krylov space for the unitary U rather than for H [1,4,5,7]. Because U is unitary and commutes with H,

$$G_{jk} = v_0^* U^{k-j} v_0 =: g_{k-j}, \qquad M_{jk} = v_0^* H U^{k-j} v_0 =: h_{k-j}, \tag{6}$$

with $g_{-k} = \overline{g_k}$ and $h_{-k} = \overline{h_k}$. Both G and M are *Hermitian Toeplitz*, so only the $2m$ numbers $g_0, \ldots, g_{m-1}$ and $h_0, \ldots, h_{m-1}$ have to be estimated. g_k is a Hadamard test with $W = U^k$. Using (1), $h_k = \sum_\ell \alpha_\ell \, v_0^* P_\ell U^k v_0$ is a weighted sum of Hadamard tests with $W = P_\ell U^k$.

Spectral picture. With the eigendecomposition from Section 4.1,

$$g_k = \sum_i |\gamma_i|^2 z_i^k, \qquad h_k = \sum_i |\gamma_i|^2 \lambda_i z_i^k, \qquad z_i = e^{-\mathrm{i}\Delta t \lambda_i}. \tag{7}$$

The data are samples of a sum of complex exponentials. This connects the method to Prony's method, matrix pencil methods and filter diagonalization in signal processing. It also shows

$$G = Z^* D Z, \qquad Z_{ik} = z_i^k, \quad D = \mathrm{diag}(|\gamma_i|^2), \tag{8}$$

so G is a weighted Gram matrix of a Vandermonde matrix with nodes on the unit circle.

Convergence. Every vector in the subspace is $p(U)v_0$ for a polynomial p of degree less than m. As for Lanczos, it follows that

$$\theta_{\min} - \lambda_0 \;\le\; (\lambda_{\max} - \lambda_0)\, \frac{1 - |\gamma_0|^2}{|\gamma_0|^2} \min_{\deg p < m} \frac{\max_{i \ge 1} |p(z_i)|^2}{|p(z_0)|^2}. \tag{9}$$

For Lanczos one has $p(\lambda)$ on the real interval $[\lambda_1, \lambda_{\max}]$ and inserts Chebyshev polynomials (Kaniel–Paige–Saad). Here p must be small on the arc of the unit circle containing $z_1, \ldots, z_{n-1}$ and large at z_0. If $\Delta t(\lambda_{\max} - \lambda_0) < 2\pi$ this arc does not wrap around, and (9) gives exponential convergence in m, with a rate governed by the angular gap $\Delta t(\lambda_1 - \lambda_0)$. Rigorous versions, including noise, are given in [6, 7]. The choice of Δt is a trade-off. If it is too large, distinct eigenvalues alias to the same z_i. If it is too small, the nodes cluster and G becomes even more ill-conditioned (this is the Lanczos limit discussed in Section 4.1). A common choice is $\Delta t \approx \pi / \|H\|_2$.

The factor $|\gamma_0|^{-2}$ in (9) matters far more than classically. A random start vector has $|\gamma_0|^2 \approx 1/n = 2^{-N}$, which is useless. One needs a preparable v_0 with non-negligible overlap with the ground state u_0. Typical choices are a product state (Section 2.1) or a single standard basis vector e_b selected by a cheap classical approximation (in chemistry the Hartree–Fock state). This *state preparation* problem is shared by all quantum eigenvalue algorithms.

4.3 Chebyshev Krylov via block encodings

With a block encoding of H/α one can prepare $v_k = T_k(H/\alpha)v_0$, and then $\mathrm{range}(V) = \mathcal{K}_m(H, v_0)$ is the *exact* Lanczos subspace [10]. Only the Chebyshev moments $\mu_k = v_0^* T_k(H/\alpha)v_0$, $k = 0, \ldots, 2m-1$, are needed, because $T_j T_k = \frac{1}{2}(T_{j+k} + T_{|j-k|})$ and $xT_k = \frac{1}{2}(T_{k+1} + T_{|k-1|})$ give

$$G_{jk} = \tfrac{1}{2}\big(\mu_{j+k} + \mu_{|j-k|}\big), \qquad M_{jk} = \tfrac{\alpha}{4}\big(\mu_{j+k+1} + \mu_{|j+k-1|} + \mu_{|j-k|+1} + \mu_{||j-k|-1|}\big), \tag{10}$$

i.e. Toeplitz-plus-Hankel matrices. Convergence is that of Lanczos, but block encodings need extra qubits and deeper circuits. This variant targets future, error-corrected hardware.

4.4 Unifying view and other variants

Both variants measure *moments* of the spectral measure $d\mu(\lambda) = \sum_i |\gamma_i|^2 \delta(\lambda - \lambda_i)$ of H with respect to v_0, in the basis $e^{-\mathrm{i}k\Delta t\lambda}$ or $T_k(\lambda/\alpha)$. Solving (4) is then the familiar problem of computing Gauss-type quadrature nodes from (modified) moments [11]. Classically this is known to be ill-conditioned in the power basis and better behaved for Chebyshev moments. The quantum setting adds random noise of size ε to the moments.

Further variants include imaginary-time bases $v_k \propto e^{-k\tau H}v_0$ (QLanczos [3], where G is Hankel up to diagonal scaling), several start vectors (block or multireference Krylov [2]), and non-Krylov subspaces. Examples are $\mathrm{span}\{v_0, P_1 v_0, \ldots, P_m v_0\}$ for selected Pauli strings or similar simple matrices P_j [12], and spans of standard basis vectors e_b whose indices b are obtained by measuring a prepared state; in the latter case $G = I$ and M is computed classically [13]. The common computational core is always the pencil (4).

5 The numerical core: a noisy, ill-conditioned pencil

Ill-conditioning is intrinsic. The better the space approximates u_0, the closer the basis is to linear dependence. By (8), G inherits the exponential ill-conditioning of Vandermonde matrices with nodes clustered on an arc, weighted by the possibly tiny $|\gamma_i|^2$. Figure 1 (right) shows $\kappa_2(G)$ growing exponentially until it reaches $1/\mathbf{u} \approx 10^{16}$ for $m \approx 20$, although n is only 1024.

Noise. The device returns $\widetilde{G} = G + E_G$ and $\widetilde{M} = M + E_M$ with $\|E_G\|, \|E_M\|/\|H\| \sim \varepsilon$, and typically $\varepsilon \in [10^{-4}, 10^{-2}]$. Once $\lambda_{\min}(G) \lesssim \varepsilon$, $\widetilde{G}$ may be indefinite and the pencil $(\widetilde{M}, \widetilde{G})$ may have eigenvalues far from the spectrum of H, possibly below λ_0. The variational guarantee $\theta_{\min} \geq \lambda_0$ is lost. Solving (4) naively gives garbage (red curve in Figure 1).

Thresholding. The standard remedy, known in quantum chemistry as canonical orthogonalization, is a truncated eigenvalue decomposition. Compute $\widetilde{G} = W\Sigma W^*$, keep the eigenpairs with $\sigma_i > \tau$ (collected in W_τ, Σ_τ), and solve the $r \times r$ standard Hermitian problem

$$\left(\Sigma_\tau^{-1/2} W_\tau^* \, \widetilde{M} \, W_\tau \Sigma_\tau^{-1/2}\right) y = \theta \, y. \tag{11}$$

This is the analogue of a truncated SVD for an ill-posed problem, with τ a few times the noise level. Why does this work at all when $\kappa(G) \gg 1/\varepsilon$? Rayleigh–Ritz only needs *one* good vector in the subspace. If $u_0 \approx Vc$ with a coefficient vector c of moderate norm, the perturbation changes its Rayleigh quotient by roughly $\varepsilon\|c\|^2$, and thresholding at τ does not remove it. Epperly, Lin and Nakatsukasa [6] turn this argument into a rigorous stability theory for quantum subspace diagonalization. The error is bounded by the noiseless Krylov error plus a term controlled by ε, τ and the size of good coefficient vectors. Kirby [7] refines the analysis for the real-time basis. In practice, the achievable accuracy is set by the noise level rather than by $\kappa(G)$.

Experiment. For Figure 1 we take the Ising chain with $N = 10$, $h = 1$, the product state $v_0 = 2^{-N/2}(1, \ldots, 1)^T = a \otimes \cdots \otimes a$ with $a = (1, 1)^T/\sqrt{2}$ as start vector ($|\gamma_0|^2 \approx 0.47$) and $\Delta t = \pi/(\lambda_{\max} - \lambda_0)$. Noise is modelled as independent complex Gaussian perturbations of g_k and h_k with standard deviations ε and $\varepsilon\,(\lambda_{\max} - \lambda_0)$, respectively, with threshold $\tau = 10\varepsilon$ (median over 50 realizations). The real-time basis converges exponentially, but here more slowly than Lanczos. Even without noise, rounding limits it to about 10^{-9} because $\kappa(G) \approx 10^{16}$. With noise, the error stagnates at roughly the noise level: about $3 \cdot 10^{-6}$ for $\varepsilon = 10^{-6}$ and $6 \cdot 10^{-4}$ for $\varepsilon = 10^{-3}$. Without thresholding there is no convergence at all.

6 Cost and practical aspects

- **Shots.** The real-time method needs $O(m)$ distinct numbers. Through (1), each h_k costs up to L Hadamard tests, and each test needs $O(\varepsilon^{-2})$ shots. Grouping commuting Pauli strings and tailoring shot budgets to entries reduce this, but sampling dominates the run time.

- **Circuit depth.** v_{m-1} requires evolution up to time $(m-1)\Delta t$. On noisy hardware the depth limits m, which favours small subspaces with a good start vector. Trotter errors mean that the implemented U commutes with H only approximately, so the Toeplitz structure of M is itself an approximation.

- **Controlled operations.** The Hadamard test needs $C(U^k)$ from (2), which is expensive. Variants avoid it when an additional vector $r \perp v_0$ with a known eigenvalue $Ur = e^{-\mathrm{i}\phi}r$ is available, e.g. from a symmetry of H; superpositions of v_0 and r then reveal $v_0^* U^k v_0$ without $C(\cdot)$.

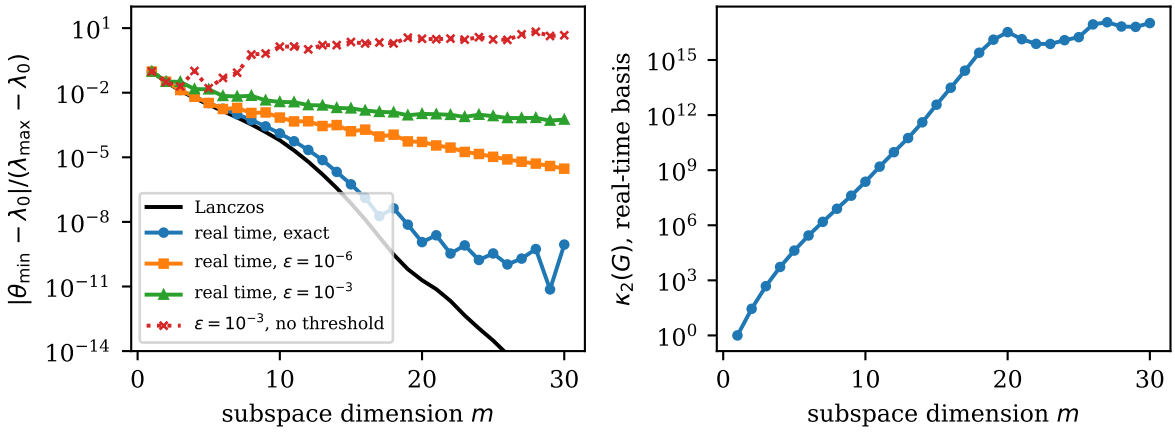


Figure 1: Ising chain, $n = 2^{10}$. Left: relative error of the smallest Ritz value for Lanczos and for the real-time Krylov method with exact and noisy matrix elements. Right: condition number of the exact Toeplitz Gram matrix G. Script: `qk_experiment.py`.

- **Comparison with phase estimation.** Quantum phase estimation, the textbook quantum eigenvalue algorithm, computes λ_0 to accuracy ε with evolution time $O(1/\varepsilon)$ and needs error-corrected hardware (Section 2.3). Krylov methods trade circuit depth for classical post-processing and many shots. They are therefore regarded as near-term methods, and they are close relatives of recent phase estimation variants designed for the first generation of error-corrected hardware, which also process the signal g_k classically.

7 Open questions for numerical linear algebra

1. **Perturbation theory for noisy definite pencils.** What are the optimal threshold τ and regularization, given a known covariance of E_G, E_M? Can structure (Hermitian Toeplitz, Hankel) be preserved or exploited, e.g. via structured low-rank approximation?

2. **Basis design.** Non-uniform time steps, filters, block Krylov spaces with several start vectors, and the trade-off between conditioning and aliasing in the choice of Δt.

3. **Moments and quadrature.** How stable is recovering Lanczos coefficients or Gauss quadrature rules from noisy trigonometric or Chebyshev moments, and what do modified-moment algorithms [11] offer?

4. **Experimental design.** Which entries should be measured, and with how many shots, for a given total budget?

5. **Beyond the ground state.** Excited states, spectral densities and quantities $v_0^* f(H) v_0$ from the same data, plus error bounds that account for Trotter errors.

For a broad survey of subspace methods on quantum computers see [14]. The analysis in [6] is written for a numerical analysis audience and is the natural next read.

References

[1] R. M. Parrish, P. L. McMahon, Quantum filter diagonalization: quantum eigendecomposition without full quantum phase estimation, arXiv:1909.08925, 2019.

[2] N. H. Stair, R. Huang, F. A. Evangelista, A multireference quantum Krylov algorithm for strongly correlated electrons, J. Chem. Theory Comput. 16 (2020), 2236–2245.

[3] M. Motta, C. Sun, A. T. K. Tan, M. J. O'Rourke, E. Ye, A. J. Minnich, F. G. S. L. Brandão, G. K.-L. Chan, Determining eigenstates and thermal states on a quantum computer using quantum imaginary time evolution, Nature Physics 16 (2020), 205–210.

[4] K. Klymko et al., Real-time evolution for ultracompact Hamiltonian eigenstates on quantum hardware, PRX Quantum 3 (2022), 020323.

[5] C. L. Cortes, S. K. Gray, Quantum Krylov subspace algorithms for ground- and excited-state energy estimation, Phys. Rev. A 105 (2022), 022417.

[6] E. N. Epperly, L. Lin, Y. Nakatsukasa, A theory of quantum subspace diagonalization, SIAM J. Matrix Anal. Appl. 43 (2022), 1263–1290.

[7] W. Kirby, Analysis of quantum Krylov algorithms with errors, Quantum 8 (2024), 1457.

[8] N. Yoshioka et al., Krylov diagonalization of large many-body Hamiltonians on a quantum processor, Nature Communications (2025); arXiv:2407.14431.

[9] A. Gilyén, Y. Su, G. H. Low, N. Wiebe, Quantum singular value transformation and beyond, Proc. STOC 2019, 193–204.

[10] W. Kirby, M. Motta, A. Mezzacapo, Exact and efficient Lanczos method on a quantum computer, Quantum 7 (2023), 1018.

[11] G. H. Golub, G. Meurant, Matrices, Moments and Quadrature with Applications, Princeton University Press, 2010.

[12] J. R. McClean, M. E. Kimchi-Schwartz, J. Carter, W. A. de Jong, Hybrid quantum-classical hierarchy for mitigation of decoherence and determination of excited states, Phys. Rev. A 95 (2017), 042308.

[13] J. Robledo-Moreno et al., Chemistry beyond the scale of exact diagonalization on a quantum-centric supercomputer, Science Advances (2025).

[14] M. Motta et al., Subspace methods for electronic structure simulations on quantum computers, Electronic Structure 6 (2024), 013001.